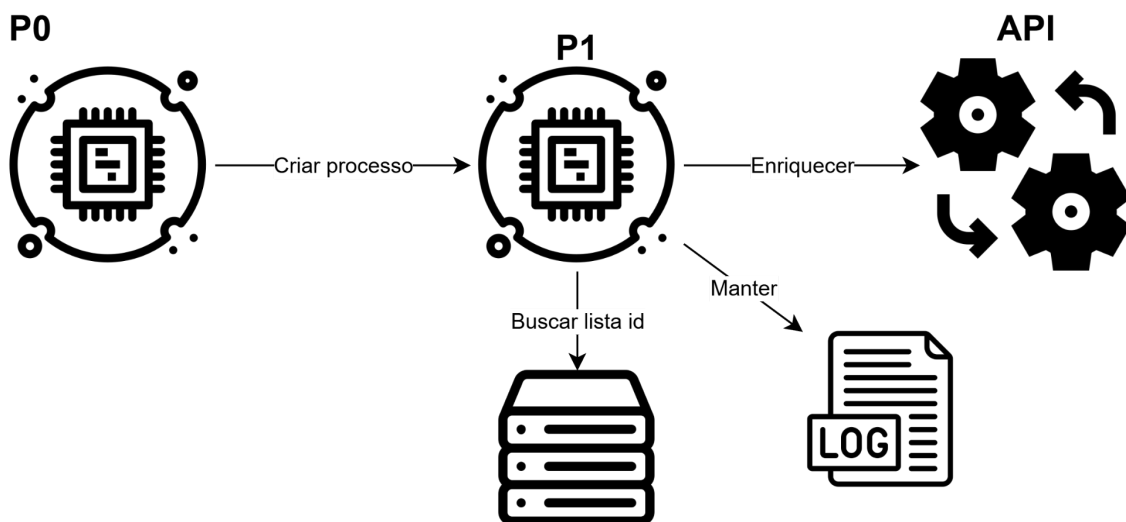


Programação Concorrente



Você deve implementar um sistema que enriquece uma lista de identificadores (`lista_ids.txt`) consultando uma **API simulada (mockada)** que devolve dados em formato JSON, registrando o resultado em um arquivo de log.

O sistema deve ser composto por dois processos:

- **P0 (processo pai):** cria o processo P1, aguarda sua finalização e verifica se ele terminou com sucesso.
- **P1 (processo filho):** cria **N threads** de trabalho, que processam os IDs da lista **em paralelo**, respeitando as regras abaixo.

Pode ser implementado em qualquer linguagem, desde que utilize mecanismos **nativos do sistema operacional** para criação de processos (ex.: `fork/exec` em C, `os.fork/subprocess` em Python, `ProcessBuilder` em Java) e para criação de threads (ex.: `pthread_create`, `threading.Thread`, `java.lang.Thread`).

Requisitos funcionais

P0 (processo pai/orquestrador) assume três responsabilidades:

1. **Orquestração multi-execução:** P0 executa P1 **duas vezes em sequência** sobre o mesmo `lista_ids.txt`, uma vez com **N = 1 thread**, outra com **N > 1 threads** (grupo escolhe N), repetindo o teste com **listas de tamanhos diferentes** (pequena, média, grande). Para cada execução, aguarda (`wait/waitpid`) e cronometra o tempo total.
2. **Auditoria de término:** ao final de cada execução, P0 verifica o código de saída de P1, diferenciando término normal, término com erro, e (se ocorrer) término por sinal, e confere se o número de linhas do log bate com o número de IDs da lista de entrada, reportando "enriquecimento incompleto" mesmo que o código de saída tenha sido 0.
3. **Relatório consolidado:** ao final de todas as execuções, P0 imprime uma tabela comparativa (N threads x tamanho da lista x tempo total x status).

P1 (processo filho/trabalhador) deve:

- Ler os IDs de `lista_ids.txt`.
- Criar **N threads** (parâmetro configurável, recebido de P0) para processar os IDs em paralelo.
- Garantir, via exclusão mútua (`mutex/semáforo`), que **cada ID seja processado exatamente uma vez**, nenhuma duplicidade na distribuição.
- Consultar a API mockada para cada ID e escrever o resultado no arquivo de log, protegendo a escrita concorrente com exclusão mútua.

3. Formato da API mockada:

Toda resposta da função mock deve seguir esta estrutura fixa (chaves e aspas em formato JSON válido):

json

```
JSON
{"id": 12345, "status": "ok", "valor": 987.65}
```

4. Formato do arquivo de log:

Cada linha deve seguir esta ordem de campos, separados por vírgula:

```
None
<data_execucao>, <thread_id>, <id_processado>, <resposta_json>
```

Exemplo:

JavaScript

```
2025-09-04 19:45:12, Thread-1, 100,  
{ "id":100, "status":"ok", "valor":987.65}  
2025-09-04 19:45:12, Thread-3, 101,  
{ "id":101, "status":"ok", "valor":543.21}
```

5. Análise de desempenho (novo requisito):

O grupo deve executar o programa duas vezes sobre o mesmo `lista_ids.txt`, uma vez com **N = 1 thread** e outra com **N > 1 threads** (o grupo escolhe o valor de N), e também deve **testar com diferentes quantidades de IDs na lista** (por exemplo, pequena, média e grande) para observar como o ganho de desempenho se comporta em diferentes escalas. Para cada execução, medir e registrar o **tempo total de execução de P1** (do início da leitura da lista até a última linha do log ser escrita). Apresentar os resultados em uma tabela simples no relatório e escrever um parágrafo curto explicando o que foi observado (ganho de tempo, algum gargalo identificado, se a exclusão mútua limitou o paralelismo em algum ponto, etc.).

Entregáveis

- Código-fonte completo e documentado no github.
- Arquivo `lista_ids.txt` de exemplo utilizado nos testes (ou script gerador).
- Arquivo(s) de log gerados nas execuções de teste.
- Relatório curto (1-2 páginas) com a tabela e análise de desempenho.